# Rapport Technique

Recherche dans un Arbre Binaire

Comparaison de performance :

BFS (Parcours en Largeur) vs std::map

**YAHYA ELMAOUI**

https://github.com/Yahya2e0016/BFS-VS-MAP.git

28 octobre 2026

## Table des matières

**1 Introduction** **2**

**2 Structure de données** **2**
2.1 Définition du nœud . . . 2
2.2 Construction de l'arbre . . . 2
2.3 Représentation visuelle (petit exemple) . . . 3

**3 Algorithmes de recherche** **3**
3.1 Recherche par BFS (Parcours en Largeur) . . . 3
3.2 Construction de la Map . . . 4
3.3 Recherche dans la Map . . . 4

**4 Résultats du Benchmark** **4**
4.1 Recherche d'un élément existant (pire cas BFS) . . . 5
4.2 Recherche d'un élément inexistant . . . 5
4.3 Coût de construction de la Map . . . 5
4.4 Graphique comparatif . . . 5

**5 Analyse des résultats** **5**
5.1 Complexité temporelle observée . . . 5
5.2 Quand utiliser chaque méthode ? . . . 6
5.3 Compromis mémoire . . . 6

**6 Conclusion** **7**

# 1 Introduction

Ce rapport présente une analyse comparative de deux méthodes de recherche dans un arbre binaire complet contenant jusqu'à 1 000 000 de nœuds :

1. **Recherche par parcours en largeur (BFS)** : parcours niveau par niveau de l'arbre à l'aide d'une file (`std::deque`).
2. **Recherche par table associative (`std::map`)** : indexation préalable de tous les nœuds dans une structure de type arbre rouge-noir, puis recherche par clé.

L'objectif est de mettre en évidence les différences de complexité algorithmique entre ces deux approches et d'observer leur impact concret sur le temps d'exécution.

# 2 Structure de données

## 2.1 Définition du nœud

Chaque nœud de l'arbre est représenté par la structure suivante :

```
struct Node {
    int id;
    std::vector<Node*> children;
    Node(int val) : id(val) {}
    bool addChild(Node* child) {
        if (children.size() >= 2) {
            std::cout << "Error: Node " << id
                      << " already has 2 children\n";
            return false;
        }
        children.push_back(child);
        return true;
    }
};
```

— `id` : identifiant entier unique du nœud.
— `children` : vecteur de pointeurs vers les nœuds enfants, limité à 2 éléments par la méthode `addChild()` pour garantir la contrainte binaire.

## 2.2 Construction de l'arbre

L'arbre binaire complet est construit par une boucle qui crée `N` nœuds numérotés de 1 à `N`, puis les relie selon la formule d'indexation d'un arbre binaire complet :

```
const int TOTAL_NODES = 1000000;
std::vector<Node*> nodes;
nodes.reserve(TOTAL_NODES);

for (int i = 1; i <= TOTAL_NODES; ++i) {
    nodes.push_back(new Node(i));
}

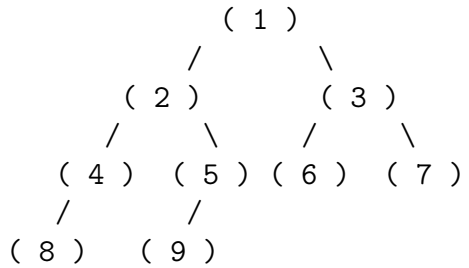
// Parent de l'indice i = (i - 1) / 2
for (int i = 1; i < TOTAL_NODES; ++i) {
```

```
    int parentIdx = (i - 1) / 2;
    nodes[parentIdx]->addChild(nodes[i]);
}

Node* root = nodes[0];
```

### 2.3 Représentation visuelle (petit exemple)

Pour 9 nœuds, la structure obtenue est :




Propriétés :

- **Enfant gauche** de l'indice $i$ : $2i + 1$
- **Enfant droit** de l'indice $i$ : $2i + 2$
- **Parent** de l'indice $i$ : $\lfloor (i - 1)/2 \rfloor$

## 3 Algorithmes de recherche

### 3.1 Recherche par BFS (Parcours en Largeur)

Le BFS utilise une file (FIFO) pour visiter les nœuds niveau par niveau. Chaque nœud est comparé à la cible avant d'enfiler ses enfants.

```
void bfsTraversal(Node* root, int target) {
    if (root == nullptr) return;
    std::deque<Node*> dq;
    dq.push_back(root);

    std::cout << "BFS Traversal: ";
    while (!dq.empty()) {
        Node* current = dq.front();
        if (current->id == target) {
            std::cout << "found " << target << "\n";
            return;
        }
        dq.pop_front();
        for (Node* child : current->children) {
            if (child != nullptr) {
                dq.push_back(child);
            }
        }
    }
    std::cout << "not found " << target << "\n";
}
```

**Complexité :**

— Temps : $O(N)$ dans le pire cas (doit parcourir tous les nœuds).
— Espace : $O(W)$ où $W$ est la largeur maximale de l'arbre ($\approx N/2$ pour un arbre complet).

### 3.2 Construction de la Map

Avant de pouvoir rechercher via `std::map`, il faut parcourir l'arbre une première fois pour indexer chaque nœud :

```
void buildMapFromTree(Node* root,
                      std::map<int, Node*>& nodeMap) {
    if (root == nullptr) return;
    std::deque<Node*> dq;
    dq.push_back(root);

    while (!dq.empty()) {
        Node* current = dq.front();
        dq.pop_front();
        nodeMap[current->id] = current;
        for (Node* child : current->children) {
            if (child != nullptr) {
                dq.push_back(child);
            }
        }
    }
}
```

**Complexité de la construction :** $O(N \log N)$ car chaque insertion dans `std::map` coûte $O(\log N)$.

### 3.3 Recherche dans la Map

```
void searchInMap(const std::map<int, Node*>& nodeMap,
                 int target) {
    std::cout << "Map Search: ";
    auto it = nodeMap.find(target);
    if (it != nodeMap.end()) {
        std::cout << "found " << target << "\n";
    } else {
        std::cout << "not found " << target << "\n";
    }
}
```

**Complexité :** $O(\log N)$ car `std::map` est implémentée en interne comme un arbre rouge-noir équilibré.

## 4 Résultats du Benchmark

Les mesures ont été effectuées avec `std::chrono::high_resolution_clock` et compilées avec `g++ -O2 -std=c++17` sur un processeur Intel Core i5-10310U (1.70 GHz, 8 threads) avec 16 Go de RAM.

### 4.1 Recherche d'un élément existant (pire cas BFS)

La cible est le dernier élément de l'arbre (valeur = $N$), forçant le BFS à parcourir la quasi-totalité des nœuds.

| **Taille $N$** | **Cible** | **BFS (ms)** | **Map (ms)** | **Ratio BFS/Map** |
|---:|---:|---:|---:|---:|
| 1 000 | 1 000 | 0.010 | 0.0003 | $\approx 31\times$ |
| 10 000 | 10 000 | 0.097 | 0.0005 | $\approx 177\times$ |
| 100 000 | 100 000 | 2.18 | 0.004 | $\approx 587\times$ |
| 500 000 | 500 000 | 6.37 | 0.005 | $\approx 1\,337\times$ |
| 1 000 000 | 1 000 000 | 11.53 | 0.004 | $\approx 3\,203\times$ |

TABLE 1 – Temps de recherche – élément existant (pire cas pour BFS)

### 4.2 Recherche d'un élément inexistant

La cible est $N + 999$, forçant les deux méthodes à examiner l'intégralité de leur espace de recherche.

| **Taille $N$** | **Cible** | **BFS (ms)** | **Map (ms)** | **Ratio BFS/Map** |
|---:|---:|---:|---:|---:|
| 1 000 | 1 999 | 0.009 | 0.0004 | $\approx 24\times$ |
| 10 000 | 10 999 | 0.088 | 0.0006 | $\approx 151\times$ |
| 100 000 | 100 999 | 1.55 | 0.003 | $\approx 460\times$ |
| 500 000 | 500 999 | 8.10 | 0.004 | $\approx 2\,271\times$ |
| 1 000 000 | 1 000 999 | 13.34 | 0.004 | $\approx 3\,511\times$ |

TABLE 2 – Temps de recherche – élément inexistant

### 4.3 Coût de construction de la Map

| **Taille $N$** | **Temps de construction (ms)** |
|---:|---:|
| 1 000 | 0.12 |
| 10 000 | 1.14 |
| 100 000 | 13.29 |
| 500 000 | 72.81 |
| 1 000 000 | 147.22 |

TABLE 3 – Temps de construction de `std::map` par parcours BFS

### 4.4 Graphique comparatif

## 5 Analyse des résultats

### 5.1 Complexité temporelle observée

— **BFS** : Le temps croît de manière **linéaire** avec $N$. En passant de $10^3$ à $10^6$ nœuds (facteur 1000×), le temps passe de $\sim 0.01\,\text{ms}$ à $\sim 11.5\,\text{ms}$ (facteur $\sim 1150\times$),

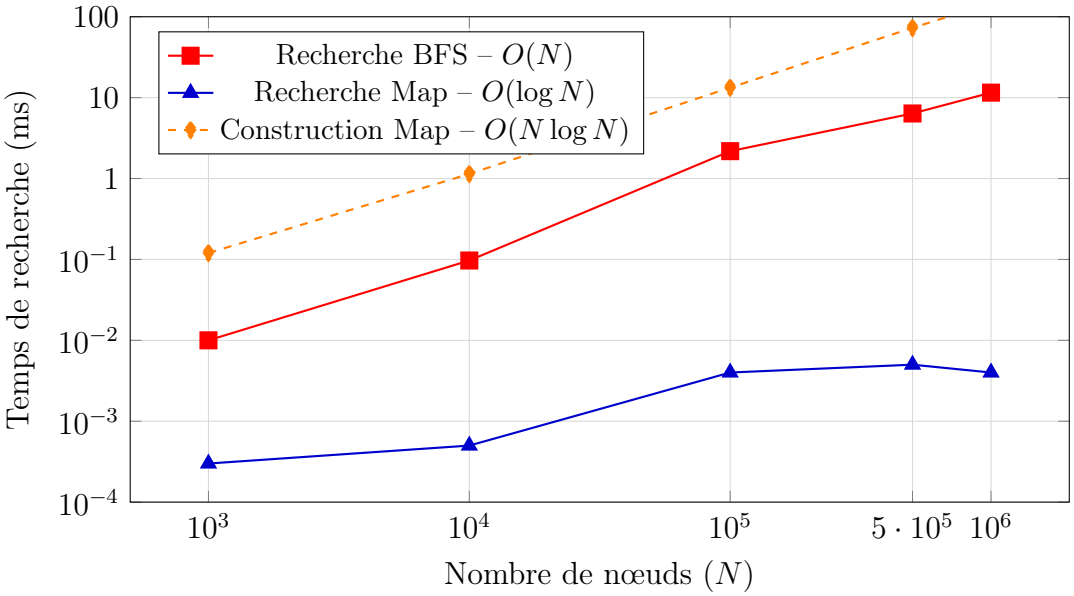



FIGURE 1 – Comparaison des temps d'exécution (échelle logarithmique)

confirmant la complexité $O(N)$.

— **Map (recherche)** : Le temps reste quasi constant autour de 0.003–0.005 ms quelle que soit la taille, ce qui est cohérent avec $O(\log N)$. En effet, $\log_2(10^6) \approx 20$, donc la Map effectue au maximum $\sim 20$ comparaisons même pour 1 million d'éléments.

— **Map (construction)** : Coût initial de $\sim 147$ ms pour $10^6$ éléments ($O(N \log N)$). Ce coût est amorti si l'on effectue de nombreuses recherches par la suite.

## 5.2 Quand utiliser chaque méthode ?

| **Utiliser BFS** | **Utiliser std : :map** |
|---|---|
| Recherche unique ou ponctuelle | Recherches fréquentes et répétées |
| Arbre de petite taille ($N < 10\,000$) | Arbre de grande taille ($N > 100\,000$) |
| Mémoire limitée (pas de structure annexe) | Mémoire suffisante pour l'index |
| Pas besoin de pré-traitement | Le coût initial de construction est acceptable |

TABLE 4 – Guide de choix entre BFS et Map

## 5.3 Compromis mémoire

— **BFS** : utilise uniquement la file temporaire ($\sim N/2$ pointeurs au maximum).

— **std : :map** : chaque entrée occupe environ 64 octets (nœud rouge-noir avec pointeurs parent/gauche/droite, couleur, clé et valeur). Pour $N = 10^6$, cela représente $\sim 64$ Mo de mémoire supplémentaire.

# 6 Conclusion

Cette étude démontre clairement la différence fondamentale entre une recherche séquentielle ($O(N)$) et une recherche indexée ($O(\log N)$) :

- Pour $N = 1\,000\,000$, la recherche par `std::map` est $\sim 3\,200$ **fois plus rapide** que le parcours BFS.
- Cet avantage augmente proportionnellement avec $N$ : plus l'arbre est grand, plus l'écart se creuse.
- Le prix à payer est un coût initial de construction ($\sim 147$ ms pour $10^6$ éléments) et une consommation mémoire supplémentaire ($\sim 64$ Mo).

En pratique, si une seule recherche est nécessaire, le BFS est suffisant. En revanche, dès que plusieurs recherches sont prévues, l'investissement dans une `std::map` (ou une `std::unordered_map` pour $O(1)$ en moyenne) devient largement rentable.

# Environnement de test

| | |
|---|---|
| **Processeur** | Intel Core i5-10310U @ 1.70 GHz (8 threads) |
| **Mémoire** | 16 Go DDR4 |
| **OS** | Linux (Arch) |
| **Compilateur** | g++ (GCC) avec `-O2 -std=c++17` |
| **Mesure** | `std::chrono::high_resolution_clock` |
| **Code source** | https://github.com/Yahya2e0016/BFS-VS-MAP.git |